

Asignatura: Fundamentos de algoritmia

Evaluación continua. Cuestionario.

Grupo A. Amarillo
Profesor: Isabel Pita.

Nombre del alumno:

1. Tenemos una máquina capaz de realizar 10^8 operaciones por segundo. Tenemos una serie de algoritmos que se ejecutan en tiempo a) $\mathcal{O}(n^2)$, b) $\mathcal{O}(n)$, c) $\mathcal{O}(n \log(n))$ y d) $\mathcal{O}(2^n)$. Indica que tamaño de entrada elegirías para cada uno de ellos si quieres que se ejecuten aproximadamente en un segundo: 1) 10, 2) 100, 3) 1.000, 4) 10.000, 5) 100.000, 6) 1.000.000, 7) 10.000.000, 8) 100.000.000

Respuesta:

- a) Si el algoritmo tiene complejidad $\mathcal{O}(n^2)$, se utiliza una entrada entre 1.000 y 10.000 datos, porque $(10^4)^2 = 10^8$.
 - b) Si el algoritmo tiene complejidad $\mathcal{O}(n)$, se utiliza una entrada entre 10.000.000 y 100.000.000 datos. Aunque normalmente no son aceptables entradas de mas de 1.000.000 por problemas de espacio.
 - c) Si el algoritmo tiene complejidad $\mathcal{O}(n \log(n))$, se utiliza una entrada entre 100.000 y 1.000.000 de datos.
 - d) Si el algoritmo tiene una complejidad $\mathcal{O}(2^n)$, se utilizan entradas de unos 100 datos.
2. Indica la complejidad en tiempo más ajustada del siguiente algoritmo, y justificala indicando el número de vueltas que da cada bucle y el coste de cada vuelta.

```
int n; std::cin >> n;
for (int i = 0; i < n; ++i)
    for (int j = n; j >= i; --j)
        x += v[i][j];
}
```

Respuesta:

El bucle externo (variable de control i) da n vueltas, siendo n el valor leído en la entrada.

En la primera vuelta del bucle externo el bucle interno (variable de control j) da n vueltas cada una de ellas de coste constante (se trata de una operación aritmética y un acceso a una matriz). En la siguiente vuelta del bucle externo el bucle interno dará $n - 1$ vueltas de coste constante etc.

Podemos considerar que el coste de la primera vuelta del bucle externo es n , el coste de la segunda vuelta es $(n - 1)$ etc. Si sumamos el coste de las n vueltas del bucle externo tenemos $n + (n - 1) + (n - 2) + \dots + 1 + 0 = \sum_{k=0}^{k=n} k = (n * (n + 1)) / 2$ que pertenece al orden $\mathcal{O}(n^2)$.

3. Indica la complejidad en tiempo más ajustada del siguiente algoritmo, y justificala indicando el número de vueltas que da cada bucle y el coste de cada vuelta.

```
int n; std::cin >> n;
for (int i = 1; i < n; i = i*2) ++x;
```

Respuesta:

La variable de control del bucle se multiplica por dos en cada vuelta del bucle, por lo que el bucle alcanza el valor n en $\log n$ vueltas. El coste de cada vuelta del bucle es constante porque consiste en incrementar una variable. Por lo tanto el coste del bucle completo pertenece al orden $\mathcal{O}(\log n)$.

4. Escribe un predicado para expresar que los valores de un vector v , entre las posiciones i y j están ordenados en orden creciente. Los valores del vector pueden ser repetidos.

Respuesta:

Se consideran incluidos los límites i y j .

$\text{creciente}(v, i, j) \equiv \forall k : i \leq k < j : v[k] \leq v[k + 1]$

Otra forma de escribir este predicado es:

$\text{creciente}(v, i, j) \equiv \forall x, y : i \leq x < y \leq j : v[x] \leq v[y]$

5. Especifica un algoritmo que dado un vector de números enteros, cuya longitud es mayor que cero, calcule la longitud de la secuencia creciente más larga.

Respuesta:

En este caso se nos pide la especificación completa del algoritmo, por lo que hay que dar la precondition y la postcondición:

$P : \{v.size() > 0\}$

`secMax (vec<int> v) dev int l`

$Q : \{l == \#p, q : 0 \leq p < q \leq v.size() \wedge creciente(v, p, q) : q - p\}$

6. Dada la especificación:

$P : \{v.size() > 0\}$

`resolver(vector<int> v) dev int sol`

$Q : \{sol == \#k : 0 \leq k < v.size() : v[k] > (max\ j : k < j < v.size() : v[j])\}$

Y dada la siguiente implementación:

```
int resolver(std::vector<int> const& v) {
    int i = v.size()-1; int cont = 0; int m = v[v.size()-1];
    while (i > 0) {
        if (v[i-1] > m) {
            m = v[i-1];
            ++cont;
        }
        --i;
    }
    return cont;
}
```

Indica un invariante del bucle que nos permita probar la corrección del mismo. Recuerda que en el invariante se debe expresar lo que se calcula en todas las variables que intervienen en el bucle, no solo en la variable que se devuelve como resultado de la función.

Respuesta:

La primera parte del invariante expresa que la variable `cont` del programa tiene como valor el número de elementos del vector, entre la posición `i` indicada por la variable de control del bucle y el final del vector, cuyo valor es mayor que el máximo de los que tiene a la derecha.

La segunda parte del invariante expresa que la variable `m` del programa tiene como valor el máximo del vector entre la posición `i` y el final del vector (`v.size()`).

$I : \{cont == \#k : 0 \leq k < v.size() : v[k] > (max\ j : i < j < v.size() : v[j]) \wedge m = (max\ j : i < j < v.size() : v[j])\}$

7. Indica cuál es la recurrencia que permite calcular el coste en el caso peor del siguiente algoritmo, haz el desplegado e indica el coste final del algoritmo.

```
int f ( std::vector<int> const & v, int ini, int fin){
    if (ini == fin) return 0; // Vector vacio
    else {
        int m = (ini + fin - 1) / 2;
        int x1 = f(v, ini, m+1);
        int x2 = f(v, m+1, fin);
        return x1 + x2;
    }
}
```

Respuesta:

Se realizan dos llamadas recursivas cada una con la mitad de los datos. El coste de la parte recursiva, sin tener en cuenta las llamadas, es constante porque se realizan únicamente operaciones aritméticas y asignaciones.

$$T(n) = \begin{cases} c_0 & \text{if } ini == fin \\ 2T(n/2) + c_1 & \text{if } ini < fin \end{cases}$$

El desplegado y el coste final se encuentran en el cuaderno de problemas de complejidad de algoritmos recursivos del campus.

8. Una función recursiva que permite resolver el problema de la búsqueda binaria es $(m = (ini + fin)/2)$ y fin el siguiente al último elemento):

$$binarySearch(v, ini, fin, x) = \begin{cases} x == v[ini] & \text{if } ini + 1 == fin \\ binarySearch(v, ini, m, x) & \text{if } ini + 1 < fin \wedge x < v[m] \\ binarySearch(v, m, fin, x) & \text{if } ini + 1 < fin \wedge v[m] \leq x \end{cases}$$

Siguiendo este ejemplo escribe una función recursiva que permita resolver el problema de calcular la suma de los valores de un vector.

Respuesta:

$$suma(v, ini, fin) = \begin{cases} 0 & \text{if } ini == fin \text{ (vector vacio)} \\ suma(v, ini + 1, fin) + v[ini] & \text{if } ini < fin \end{cases}$$

La llamada inicial a la función es $suma(v, 0, v.size())$;

Otras variantes son:

$$suma(v, ini, fin) = \begin{cases} 0 & \text{if } ini == fin \text{ (vector vacio)} \\ suma(v, ini, fin - 1) + v[fin] & \text{if } ini < fin \end{cases}$$

o

$$suma(v, ini, fin) = \begin{cases} 0 & \text{if } ini == fin \text{ (vector vacio)} \\ suma(v, ini, m) + suma(v, m, fin) & \text{if } ini < fin \wedge m = (ini + fin)/2 \end{cases}$$

9. De los algoritmos de ordenación que conoces: inserción, selección, burbuja, mergesort y quicksort, indica cual de ellos es más eficiente para ordenar un vector cuyos elementos están casi ordenados. Indica el orden de complejidad del algoritmo en el caso peor (elementos desordenados) y en el caso mejor (elementos casi ordenados).

Respuesta:

Si los elementos están casi ordenados el algoritmo más eficiente es el de inserción. Cuando los valores están completamente ordenados, el algoritmo de inserción recorre el vector una única vez, por lo que su coste es lineal respecto al número de elementos del vector en el caso mejor.

En el caso peor, que ocurre cuando los elementos del vector están ordenados en el orden contrario al que se pide o bien están simplemente *bastante* desordenados, el coste del algoritmo de inserción es cuadrático respecto al número de elementos del vector. Esto se debe a que cada elemento del vector se debe dejar caer hasta su posición en la parte ya ordenada. En el caso peor cada elemento se debe intercambiar con todos los elementos que están en posiciones anteriores a él.

10. De los algoritmos de ordenación que conoces: inserción, selección, burbuja, mergesort y quicksort, indica cual de ellos es más eficiente para ordenar un vector cuyos elementos son números aleatorios. Indica el orden de complejidad del algoritmo en el caso peor y en el caso mejor.

Respuesta:

Cuando los elementos del vector están desordenados el mejor algoritmo para ordenarlos es el quicksort. Tiene complejidad $\mathcal{O}(n \log n)$ en el caso medio y complejidad $\mathcal{O}(n^2)$ en el caso peor.

Este algoritmo se comporta mejor en la practica que el algoritmo del mergesort (que tiene complejidad en el caso peor del orden de $\mathcal{O}(n \log n)$) debido a la alta constante multiplicativa del algoritmo del mergesort derivada de que la mezcla ordenada se realiza en un vector auxiliar que luego hay que copiar en el vector principal.